

STANDARD DE SÉCURITÉ APPLICATIVE — FLEXWORK_IOS

Un modèle en cinq couches, et les trois trous qui restent

L'audit du positionnement des pages a mis au jour une architecture de routes saine mais une autorisation appliquée *route par route*, à la main. Ce document décrit le modèle standard à mettre en place : ce qui est déjà solide, ce qui manque réellement — trois failles vérifiées dans le code, pas des hypothèses — et le contrat de garde unique qui doit remplacer les contrôles ad hoc.

86 routes API analysées 3 failles confirmées 1 couche entièrement absente 0 route non authentifiée par erreur

01 LE MODÈLE CIBLE

Cinq couches traversées dans l'ordre par chaque requête. Chacune a un rôle unique et ne suppose jamais que la précédente a fait son travail — c'est la défense en profondeur : une couche contournée ne doit pas suffire à atteindre la donnée.



AUTORISATION rôle, statut KRC, et surtout rien de propriété entre l'appelant et l'objet visé.

PARTIELLE



C4 ·

INTÉGRITÉ

La charge utile est-elle valide et infalsifiable ?

Validation Zod systématique, chaînage par hash des déclarations, contrats et journaux.

SOLIDE



C5 ·

TRAÇABILITÉ

Qui a fait quoi, et peut-on le prouver ?

Journal d'audit append-only avec justification obligatoire sur toute action administrative.

SOLIDE

02 LES FAILLES CONFIRMÉES

Chacune a été vérifiée dans le code source, pas déduite d'un motif. Toutes appartiennent à la même classe : la route **authentifie** correctement l'appelant, puis lui sert une ressource désignée par un identifiant d'URL **sans vérifier qu'il a un lien avec elle**. C'est le motif IDOR — la faille la plus courante des API par ressource.

F-01 Les candidatures de n'importe quelle mission sont lisibles

```
GET /api/missions/[id]/proposals
```

La route vérifie qu'une session existe, puis retourne toutes les candidatures de la mission demandée. Elle ne compare jamais `mission.clientId` à l'utilisateur connecté. Tout compte authentifié — y compris un prestataire concurrent — peut donc lire l'intégralité des offres reçues par un client sur une mission qui ne le concerne pas.

CE QUI FUT

Montants proposés par chaque concurrent, messages de candidature, identité et note moyenne des prestataires, nombre d'offres déjà envoyées. En clair : de quoi sous-coter systématiquement toutes les candidatures de la plateforme.

F-02 Les métadonnées de signature de tout contrat sont lisibles

F-02 Les métadonnées de signature de tout contrat sont publiques

POST /api/signature/verify

La route accepte un `contractId` arbitraire dans le corps de la requête et retourne les enregistrements de signature associés, sans vérifier que l'appelant est partie à ce contrat. Le contrôle de session est présent ; le contrôle de propriété est absent.

CE QUI FUT

Nom et adresse e-mail des signataires, empreinte de clé publique (`keyFingerprint`), empreinte des données signées, horodatage — pour n'importe quel contrat de la plateforme, par simple énumération d'identifiants.

F-03 Un appel visio peut être déclenché vers n'importe qui

POST /api/webrtc/signal

La route relaie un signal WebRTC vers l'utilisateur désigné par le champ `to`, sans vérifier qu'un lien réel (mission partagée, candidature, contrat) existe entre l'émetteur et le destinataire. Le destinataire reçoit l'événement par SSE et son interface se met à sonner.

CE QUI EST POSSIBLE

Faire sonner n'importe quel utilisateur connecté à volonté, découvrir sa présence en ligne, et l'exposer à des sollicitations hors de tout cadre contractuel — un vecteur de harcèlement, pas seulement une nuisance.

F-04 Aucune limitation de débit globale

— aucun `middleware.ts` dans le projet —

La limitation de débit n'existe qu'à quatre endroits isolés : demande d'OTP et quotas de validation KYC. Les 82 autres routes n'ont aucun plafond. Il n'existe pas de fichier `middleware.ts`, donc aucun point de passage obligatoire avant l'exécution des routes.

CONSÉQUENCE DIRECTE

Les failles F-01 et F-02 deviennent exploitables *à l'échelle* : rien n'empêche d'énumérer méthodiquement tous les identifiants de mission ou de contrat pour aspirer la base. Sans C1, une faille d'autorisation cesse d'être ponctuelle et devient une extraction massive.

03 LE CONTRAT DE GARDE UNIQUE

La cause racine n'est pas trois oublis indépendants : c'est que **chaque route décide elle-même** comment vérifier la propriété. Le projet possède déjà deux gardes partagés — `requireAdmin()` pour le rôle et `requireVerifiedKyc()` pour le statut — mais aucun pour la relation entre un utilisateur et une ressource. C'est précisément le garde manquant.

Le principe

Toute route qui reçoit un identifiant de ressource dans son URL ou son corps doit obtenir cette ressource *via un garde qui inclut la condition de propriété dans la requête elle-même*, et jamais la charger puis vérifier après coup. Une ressource dont l'appelant n'est pas partie ne doit pas être distinguable d'une ressource inexistante : dans les deux cas, 404 — sans quoi la réponse elle-même devient un oracle d'existence.

```
// src/lib/resource-guard.ts — le garde manquant, à créer

export async function requireMissionParty(missionId: string, userId: string) {
  // La condition de propriété est DANS le where, pas dans un if après coup :
  // une mission dont l'appelant n'est ni client ni candidat n'est simplement
  // jamais retournée — indistinguable d'une mission inexistante.
  const mission = await prisma.mission.findFirst({
    where: {
      id: missionId,
      OR: [
        { clientId: userId }, // le client propriétaire
        { proposals: { some: { providerId: userId } } } // un candidat de cette mission
      ],
    },
  });
  if (!mission) return { ok: false, status: 404 } as const;
  return { ok: true, mission, isOwner: mission.clientId === userId } as const;
}
```

Décliné de la même façon pour les autres racines de propriété : `requireContractParty()`, `requireOfferRecipient()`, `requireProfileOwner()`. Trois à quatre fonctions couvrent l'intégralité de la surface.

L'application aux trois failles

Le même garde résout F-01 et F-02 sans logique nouvelle. F-03 relève de la même idée appliquée à une relation plutôt qu'à une ressource : avant de relayer un signal, vérifier qu'une mission lie réellement l'émetteur au destinataire.

```
// F-01 – proposals : le client seul voit les candidatures reçues
const guard = await requireMissionParty(missionId, userId);
if (!guard.ok) return NextResponse.json({ error: "not_found" }, { status: 404 });
if (!guard.isOwner) return NextResponse.json({ error: "not_found" }, { status: 404 });

// F-02 – signature/verify : partie au contrat uniquement
const contract = await prisma.prestationContract.findFirst({
  where: { id: contractId, OR: [{ clientId: userId }, { providerId: userId }] },
});
if (!contract) return NextResponse.json({ error: "not_found" }, { status: 404 });

// F-03 – webrtc/signal : une relation de mission doit exister
const linked = await prisma.mission.findFirst({
  where: { OR: [
    { clientId: userId, proposals: { some: { providerId: to } } },
    { clientId: to, proposals: { some: { providerId: userId } } },
  ]},
  select: { id: true },
});
if (!linked) return NextResponse.json({ error: "forbidden" }, { status: 403 });
```

À noter sur F-03 : ici le 403 est justifié plutôt que le 404, car l'existence du destinataire n'est pas un secret — c'est la *relation* qui est refusée.

04 RÉTABLIR LA COUCHE PÉRIMÈTRE

C'est la seule couche entièrement absente, et celle qui change la gravité de toutes les autres. Le projet voisin `flexwork_job` en possède déjà une implémentation éprouvée — limitation par IP avec fenêtre glissante et nettoyage périodique — qui peut servir de base plutôt que d'être réécrite.

```
// middleware.ts – à créer à la racine du projet

export const config = {
  // Exclut les fichiers statiques : le plafond doit protéger l'API et les
  // pages, pas compter les requêtes d'images et de polices.
  matcher: ["/((?!next/static|next/image).*)"]
```

```

matcher. [ /((?!_next/static|_next/image|favicon.ico).*) ],
};

export function middleware(req: NextRequest) {
  const ip = req.headers.get("x-forwarded-for")?.split(",")[0] ?? "unknown";

  // Plafond bas et distinct sur les points sensibles à l'énumération –
  // c'est exactement là que F-01 et F-02 se transforment en aspiration.
  const sensitive = req.nextUrl.pathname.startsWith("/api/signature")
    || req.nextUrl.pathname.includes("/proposals");

  if (!allow(ip, sensitive ? 20 : 60)) {
    return new NextResponse("Too Many Requests", { status: 429 });
  }

  const res = NextResponse.next();
  res.headers.set("X-Content-Type-Options", "nosniff");
  res.headers.set("X-Frame-Options", "DENY");
  res.headers.set("Referrer-Policy", "strict-origin-when-cross-origin");
  return res;
}

```

Réserve à connaître : un compteur en mémoire ne survit ni au redémarrage ni à la mise à l'échelle horizontale. C'est suffisant en développement et sur une instance unique ; en production multi-instance il faudra un magasin partagé.

05 LES SIX RÈGLES DU STANDARD

Formulées pour être vérifiables en revue de code : chacune se répond par oui ou non en lisant la route, sans avoir à raisonner sur le reste du système.

R01 La propriété se prouve dans la requête, pas après

Toute lecture ou écriture d'une ressource désignée par un identifiant externe passe par un `findFirst` dont le `where` contient la condition de propriété. Jamais un `findUnique` suivi d'un `if` — un oubli de ce `if` est invisible, un oubli de clause `where` l'est beaucoup moins.

R02 Ne pas confirmer l'existence de ce qu'on refuse

Une ressource à laquelle l'appelant n'a pas droit répond 404, pas 403. Le 403 est réservé aux cas où l'existence est déjà publique et où seule l'action est refusée. Le projet applique déjà ce principe pour l'OTP (`OtpRequestAttempt` évite l'oracle d'énumération de comptes) — il doit être généralisé.

R03 Un garde partagé, jamais une vérification recopiée

Toute condition de propriété appliquée par plus d'une route vit dans `src/lib/` et est importée. Le projet le fait déjà correctement pour `checkCandidatureEligibility()` et `acceptProposal()` ; la propriété de ressource doit rejoindre cette liste.

R04 Le client ne fait pas foi

Aucune décision d'autorisation ne repose sur une valeur envoyée par le navigateur. L'identité vient toujours de la session serveur, jamais d'un champ du corps de requête — règle déjà respectée partout, à maintenir explicitement.

R05 Toute route a un plafond

Le plafond global du middleware s'applique par défaut ; les routes sensibles à l'énumération ou coûteuses en déclarent un plus bas. Une route sans plafond est un choix à justifier, pas un état par défaut.

R06 Une route désactivée reste répondante

Une route retirée du produit renvoie 410 Gone avec la raison, plutôt que d'être supprimée silencieusement. Le projet le fait déjà pour l'ancienne route de signature — c'est la bonne pratique, à conserver comme règle.

06 CONFORMITÉ PAR DOMAINE

État de la couche C3 sur les principaux groupes de routes. Les domaines conformes le sont parce que leurs requêtes portent déjà la condition de propriété — le motif à généraliser existe donc déjà dans le projet, il suffit de l'étendre.

DOMAINE	IDENTITÉ C2	PROPRIÉTÉ C3	OBSERVATION
/api/offers/[id]/*	Oui	Conforme	findFirst filtré sur providerId — le motif de référence.
/api/profile/garants/[id]	Oui	Conforme	Portée par profile: { userId } dans le where.

/api/messages	Oui	Conforme	Contrôle de participation à la mission avant lecture et écriture.
/api/missions/[id]/contract	Oui	Conforme	Comparaison explicite mission.clientId $\neq$ userId à la génération.
/api/admin/*	Oui	Conforme	Double garde rôle + granularité par adminRole, journalisée.
/api/missions/[id]/proposals	Oui	Manquante	F-01 — aucune comparaison au client propriétaire.
/api/signature/verify	Oui	Manquante	F-02 — contractId accepté sans lien vérifié.
/api/webrtc/signal	Oui	Manquante	F-03 — destinataire arbitraire, aucune relation exigée.
/api/cron/*, /api/webhooks/*	Secret partagé	Sans objet	Protégées par jeton porteur / signature — correct pour du machine-à-machine.

07 SÉQUENCE DE MISE EN ŒUVRE

Ordonnée par ratio gravité / effort. Les deux premières étapes ferment l'essentiel du risque en quelques lignes chacune ; les suivantes rendent la régression difficile.

01

Fermer F-01 et F-02

Ajouter la condition de propriété dans les deux where concernés.

IMMÉDIAT

02	Fermer F-03 Exiger une mission liant émetteur et destinataire avant tout relais de signal.	IMMÉDIAT
----	--	----------

03	Créer <code>src/lib/resource-guard.ts</code> Extraire les conditions de propriété en gardes partagés et y rebrancher les routes.	COURT
----	--	-------

04	Créer <code>middleware.ts</code> Rétablir la couche C1 : plafond par IP et en-têtes de sécurité.	COURT
----	--	-------

Tester la propriété, pas seulement le cas nominal